- Load Balancing in GKE
- Load Balancing in GKE
- Internal vs external load balancers
- Cloud Armor
- Traffic Director in GKE
- Cloud Armor
- Configure Load Balance for GKE application
- Cloud Armor security policy

Cloud Run / Cloud Functions & Backends
Cloud Run (fully managed Containers)
- Cloud Run Functions
- Automating with triggers
- Cloud Run Functions Triggers
- What to use Cloud Run vs GKE
- Cloud Run Jobs
- Deploy microservices to Cloud Run
- Cloud Run Functions overview

Cloud Functions
- Event-driven architecture
- Cloud Functions Management/monitoring
- Integration with PubSub
- Cloud Run
- Create and deploy Cloud Function
- Trigger functions via PubSub message

Cloud Pub/Sub
- Cloud Pub/Sub messaging concepts
- Topics, subscriptions, message delivery
- Pub/Sub message ordering
- Pub/Sub dead letter topics
- Monitor Pub/Sub usage and subscriptions
- Publish and consume messages from services

Cloud Storage
- Organizing Cloud Storage buckets
- Cloud Storage Lifecycle management
- Cloud storage versioning
- Cloud Storage Pub/Sub notifications
- Cloud Storage CMEK
- Set up cross-project storage

- GKE Multi-cluster Ingress
- Multi-cluster Ingress
- Deploy Multi-cluster Ingress
- Multi-cluster networking
- Multi-cluster Ingress concepts
- Multi-cluster networking

Cloud Load Balancing
- Cloud Load Balancing Overview
- Without load balancing
- One Google anycast IP address
- Cloud Load Balancing
- Application Delivery Automation

Global load balancing
- Global load balancing
- Cloud HTTP/HTTPS Load Balancing
- Cloud HTTP/HTTPS Load Balancing
- Cloud Load Balancing Overview

Types of Load Balancers in GKE

- External Load Balancer (Internet-facing) & Internal Load Balancer (Private)
- External Load Balancer
- Internal Load Balancer (private access)
- Use Pub/Sub subscriptions with Cloud Run
- Use Pub/Sub subscriptions with Cloud Run

Cloud Load Balancer (Cloud Load Balancer) & Cloud Armor

- Internal Load Balancer (Private)
- Cloud Load Balancer (Cloud Load Balancer) (HTTP)
- Cloud Armor (WAF & DDoS Protection)
- Backend Service + Health Checks + Firewall Rules

- Cloud Load Balancing Components
- Backend Service Components
- Backend
- Backend Service
- Health Check
- Forwarding Rule
- Target HTTP(S) Proxy
- URL Map
- Frontend
- Load Balancer

- Backend Service
- Health Check
- Forwarding Rule
- Target HTTP(S) Proxy
- URL Map
- Frontend
- Load Balancer

2. Ingress (Advanced Routing)
- Ingress
- Ingress annotations
- cloud.google.com/neg: "{}"
- Internal Load Balancer (GCP L7 HTTPS LB)
- Internal Application Load Balancer (Cloud Armor)
- Ingress (GKE)
- Ingress (Cloud Armor)
- Cloud Armor (WAF & DDoS Protection)
- Ingress (Cloud Armor)

Client (Browser)
- Internal Load Balancer (GCP L7 HTTPS LB)
- Internal Application Load Balancer (Cloud Armor)
- Ingress (Cloud Armor)
- Cloud Armor (WAF & DDoS Protection)
- Ingress (Cloud Armor)
- Backend Service
- Health Check
- Forwarding Rule
- Target HTTP(S) Proxy
- URL Map
- Frontend
- Load Balancer

- Traffic Flow in GKE:
- Internal Traffic Flow

Internal Kubernetes Service
- Service
- Pods

Public Internet

- Gke Scenario
- Scenario Booking Application
- Cloud Run
- Cloud SQL (PostgreSQL)
- Mega-resource-creation
- Scenario
- Ingress
- Internal Ingress Rule
- Use Load Balancer / Ingress

Rules
- Path-based routing
- Host-traffic distribution
- Cloud Armor Integration

- Case Study Enterprise-scale E-commerce Platform
- Architecture

Backend: Internal LB / Ingress
Backend: Internal LB
Backend: Internal LB

Security
- Identities-isolation of users
- Cloud Armor-application protection
- Cloud Isolation-between services

Backend	Backend Service/Ingress
Frontend	Internal - Private Network
Backend	Internal Load Balancer
Backend	Cloud Armor Policy Attached
Backend	Internal Load Balancer
Backend	Cloud SQL Private IP

Step 1 - Deploy Application
kubectl create deployment my-app --image=nginx

Step2 - Expose as LoadBalancer
kubectl expose deployment my-app --type=LoadBalancer --port=80

kubectl expose deployment my-app
--type=LoadBalancer \
--port=80 \
--target-port=80

Step3. Get External IP
kubectl get services

EXTERNAL IP: <assigned-ip>

Step4. Test Access
curl http://<EXTERNAL IP>
Open browser: http://<EXTERNAL IP>

Pods Pods

Pod1
- kubectl scale deployment/my-app --replicas=3

Pod2
- Verify Load Balancing
- kubectl get pods -o wide

Pod3
- Refresh browser multiple times → traffic distributed

Check the logs - Check traffic distribution
kubectl logs <pod name>

CLOUD RUN (SERVERLESS CONTAINERS)

Cloud Run is a fully managed serverless platform to run containers without managing infrastructure.

Architecture

- Client
- HTTPS
- CDN(Optional)
- Cloud Run Service
- Container Instance
- Scaling

- Why it's needed :
- Serverless Containers

With Cloud Run:
- No VMs
- No clusters
- No Kubernetes configs

- Cloud Run Functions / Jobs
- Cloud Run Service

Automatic Scaling (Scale to Zero)
- Scales from 0 thousands of requests
- Pay only for actual usage
- If no traffic → scale to zero
- Cloud Run Service

- Pay only for what you use
- No idle VMs
- No clusters
- No Kubernetes configs

- Cloud Run Functions / Jobs
- Cloud Run Service

Cloud Run Functions / Jobs
- Cloud Run Functions
- Cloud Run Jobs
- Cloud Run Service

Cloud Run Service
- Cloud Run Service
- Container Instance
- Scaling

- Why it's needed :
- Serverless Containers

With Cloud Run:
- No